

High-probability bounds for Non-Convex Stochastic Optimization with Heavy Tails

Optimization in Data Analysis Project Report

Piotr Jurczyk
Jan Wyrzykowski
Tomasz Żochowski

June 2026

Abstract

In this project, we analyze two algorithms used for large transformer model training and compare them to known optimizers. Furthermore, we check the importance of using burn-in and warmup techniques for gradient stabilization during the initial phase of training. We expand on the original authors' paper by evaluating performance of these methods for a smaller transformer model.

Contents

1	Introduction	3
2	Problem Description	3
3	Solutions	4
3.1	Mathematical background	4
3.1.1	Momentum	4
3.1.2	Gradient clipping	5
3.1.3	Normalized step	5
3.2	Algorithms	6
3.2.1	Alg1	6
3.2.2	Alg2	6
3.2.3	Adam	7
3.3	Learning rate scheduling	7
4	Experiments	8
4.1	BERT	8
4.2	GPT	9
4.2.1	SmallGPT	9
4.2.2	MediumGPT	9
4.2.3	WikiText Dataset	10
4.2.4	Grid search	10
4.2.5	Burn-in and warmup comparison	10
4.2.6	Artificial heavy-tail data	10
5	Results	11
5.1	BERT	11
5.2	GPT	12
5.2.1	SmallGPT baseline	12
5.2.2	MediumGPT baseline	13
5.2.3	Grid search	13
5.2.4	Burn-in and warmup comparison	14
5.2.5	Artificial heavy-tailed data	15
6	Conclusions	18
	Appendices	20
A	Grid search plots	20
B	Burn-in and warmup plots	24

1 Introduction

Stochastic gradient descent is considered a simple, robust, and universal optimization algorithm that can be used for various machine learning tasks. In expectation, it finds (local) optimal points for both convex and non-convex functions. Furthermore, if we assume bounded gradient variance, the algorithm has a definite convergence rate; if we also assume that the distribution of gradients has light tails, SGD with momentum works with high probability.

These assumptions, however, are not certain. It turns out that, particularly in transformers, the gradients have heavy tails and their variance is practically infinite. There are certain methods utilized to fight these unfavourable properties, one of them being gradient clipping. Moreover, momentum helps with gradient stabilization, which makes the algorithm more robust.

In [3] authors describe these methods and propose two algorithms that employ them and achieve satisfying convergence rates in high probability. Furthermore, they introduce burn-in – a process of skipping initial training steps before the gradient is accumulated – and compare it to widely known warmup technique.

Our goal is to implement these algorithms and compare them to the Adam optimizer – a state-of-the-art solution for neural networks. Moreover, we check how burn-in and warmup strategies work for all three optimizers mentioned. We expand on authors’ results by applying these algorithms and learning rate manipulation strategies to much smaller transformers than BERT-Large.

2 Problem Description

The standard approach for training large machine learning models is to minimize a stochastic optimization problem formulated as

$$F(\vec{w}) = \mathbb{E}_z[f(\vec{w}, z)]$$

using first-order methods like Stochastic Gradient Descent. Traditional convergence guarantees for SGD heavily rely on the assumption that the stochastic gradients $\nabla f(\vec{w}, z)$ possess light tails or bounded variance, satisfying

$$\mathbb{E}[\|\nabla f(\vec{w}, z)\|^2] \leq G^2$$

for some G and all $\vec{w}$. However, recent empirical studies reveal that these assumptions are too optimistic for modern deep learning, as large architectures like transformers and attention models frequently generate heavy-tailed gradients.

Mathematically, heavy-tailed gradients are characterized by an extremely large or even infinite variance. Consequently, instead of a bounded second moment, these gradients only possess a bounded p -th moment for some $p \in (1, 2]$. Formally, the heavy-tailed setting assumes that the gradient noise is restricted by

$$\mathbb{E}[\|\nabla f(\vec{w}, z) - \nabla F(\vec{w})\|_*^p] \leq G^p.$$

Heavy-tailed noise introduces severe challenges in the optimization of deep neural networks. When stochastic gradient noise exhibits heavy tails, standard Stochastic Gradient

Descent (SGD) frequently produces extreme, outlier gradient estimates. This volatility destabilizes the training process and makes the theoretical convergence properties of the optimizer incredibly difficult to characterize and control.

Historically [8], theoretical analyses of non-convex SGD under heavy-tailed conditions have only yielded in-expectation convergence guarantees. In practical Deep Learning applications, however, such bounds are highly unsatisfying. Because the measured function value or gradient norm is itself a heavy-tailed random variable, an average-case mathematical guarantee offers no protection against catastrophic failures in reality. In practice, this means that while the algorithm might converge "on average" across infinitely many hypothetical trials, an individual training run can still be of exceptionally poor quality or diverge completely.

Given the massive computational expense and time required to train modern neural networks, relying on average-case success is insufficient. It is critical that the optimization process successfully finds an ε -critical point with high probability on every single run.

3 Solutions

There are a few modifications of standard gradient descent that allow to reduce negative impact of heavy-tailed gradients and that are used in many optimization algorithms. In this section, we provide formulas for these modifications, describe their impact, and show how they are combined in the algorithms we perform experiments with.

3.1 Mathematical background

3.1.1 Momentum

Momentum is one of the most common methods of stabilizing convergence in gradient descent. Its goal is to speed up convergence by including previous update in the next one. This way, in each step, geometrically reduced gradients from all previous steps are included in the calculation. This technique is used for speeding up the descent process (since accumulated gradients approximate the true gradient better than the one calculated in the current step only), but it also has stabilizing properties – a high gradient value in one iteration doesn't immediately lead to a step in the wrong direction.

Let L be the loss function, $\alpha \in (0, 1)$ be the momentum parameter (determining how quickly accumulated gradients decay), and $\varepsilon > 0$ be the learning rate. Furthermore, let $\vec{v}_t$ be a vector of accumulated gradients from previous t steps and $(\vec{x}_i, y_i)_{i=1}^m$ be a minibatch chosen at step $t + 1$. Then the update with momentum is defined as follows:

$$\begin{aligned}\vec{v}_{t+1} &= \alpha \vec{v}_t - \varepsilon \left(\frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(y_i, \vec{x}_i, \vec{\theta}_t) \right) \\ \vec{\theta}_{t+1} &= \vec{\theta}_t + \vec{v}_{t+1}\end{aligned}$$

This version of momentum was introduced in 1964 by Polyak [7]. There is also a second version introduced in 1983 by Nesterov [6], which includes calculating the gradient using pa-

rameters already updated with the accumulated gradients. It can be defined in the following way:

$$\begin{aligned}\vec{v}_{t+1} &= \alpha \vec{v}_t - \varepsilon \left(\frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(y_i, \vec{x}_i, \vec{\theta}_t + \alpha \vec{v}_t) \right) \\ \vec{\theta}_{t+1} &= \vec{\theta}_t + \vec{v}_{t+1}\end{aligned}$$

Importantly, in the original paper [3], authors use a different version of momentum:

$$\begin{aligned}\vec{v}_{t+1} &= \alpha \vec{v}_t + (1 - \alpha) \left(\frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(y_i, \vec{x}_i, \vec{\theta}_t) \right) \\ \vec{\theta}_{t+1} &= \vec{\theta}_t - \varepsilon \vec{v}_{t+1}\end{aligned}$$

This way, the learning rate does not interfere in the gradient accumulation process.

3.1.2 Gradient clipping

One way to reduce the impact of heavy tailed gradients is to clamp the maximum norm of the gradient in each step. This technique is called *gradient clipping* and is frequently used to stabilize convergence.

With previous notation in power, let $\tau > 0$ be the clipping parameter (determining the upper bound on gradient norm). For the sake of simplicity, let $\nabla L = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(y_i, \vec{x}_i, \vec{\theta}_t)$. Then, at iteration $t + 1$ the gradient used in the update step is calculated as follows:

$$\vec{g}_{t+1} = \frac{\nabla L}{\|\nabla L\|} \min(\tau, \|\nabla L\|)$$

This way, if $\|\nabla L\| \leq \tau$, then the gradient remains unchanged; and conversely, if $\|\nabla L\| > \tau$, vector $\vec{g}_{t+1}$ preserves the direction of ∇L , but its norm is equal to τ .

This technique is used before momentum – we want to normalize the gradient in the current step, not the accumulated one.

3.1.3 Normalized step

In some settings, a version of SGD called *normalized SGD* is used. The only difference from standard SGD is that, at the end, the step is performed in relation to the unit vector with the same direction as the gradient.

Let $\vec{g}_{t+1}$ be the descent direction used in step t after applying other techniques like gradient clipping and momentum and let $d(\vec{v}) = \frac{\vec{v}}{\|\vec{v}\|}$ for any vector v . Then, a normalized SGD step can be defined according to formula

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \varepsilon d(\vec{g}_{t+1}).$$

Consequently, the magnitude of the descent is controlled only by the learning rate. This requires learning rate scheduling to achieve convergence, which we discuss in detail in Section 3.3. The authors from the original paper discuss in [2] how this method doesn't guarantee

convergence when small batch size is used. Nevertheless, it is used as a way of speeding up the neural network training process.

3.2 Algorithms

The three main algorithms we focused on in this project were Alg1 and Alg2 from the main paper [3] and Adam. Let's get an overview of each one.

3.2.1 Alg1

Algorithm 1 Normalized SGD with Clipping and Momentum

Input: Initial Point $\vec{w}_1$, learning rate η , momentum parameter β , clipping parameter τ , time horizon T :
Set $\vec{m}_0 = 0$.
for $t = 1 \dots T$ **do**
 Sample $z_t \sim P_z$.
 Set $\vec{g}_t^{\text{clip}} = \frac{\nabla f(\vec{w}_t, z_t)}{\|\nabla f(\vec{w}_t, z_t)\|_*} \min(\tau, \|\nabla f(\vec{w}_t, z_t)\|_*)$.
 Set $\vec{m}_t = \beta \vec{m}_{t-1} + (1 - \beta) \vec{g}_t^{\text{clip}}$.
 Set $\vec{w}_{t+1} = \vec{w}_t - \eta d(\vec{m}_t)$.
end for

This is a variant of stochastic gradient descent designed for heavy-tailed gradients. At each step, it draws a fresh sample, computes the gradient at the current point, and then clips that gradient's norm to a threshold τ while normalizing. The clipped gradient is fed into a moving average to form a momentum $\vec{m}_t$, and the parameter update moves in the direction given by $\vec{m}_t$.

3.2.2 Alg2

Algorithm 2 NIGT with Clipping

Input: Initial Point $\vec{w}_1$, learning rate η , momentum parameter β , clipping parameter τ , time horizon T :
Set $\vec{m}_0 = 0$.
for $t = 1 \dots T$ **do**
 Sample $z_t \sim P_z$.
 $\vec{x}_t = \vec{w}_t + \frac{\beta(\vec{w}_t - \vec{w}_{t-1})}{1 - \beta}$
 Set $\vec{g}_t^{\text{clip}} = \frac{\nabla f(\vec{x}_t, z_t)}{\|\nabla f(\vec{x}_t, z_t)\|_*} \min(\tau, \|\nabla f(\vec{x}_t, z_t)\|_*)$.
 Set $\vec{m}_t = \beta \vec{m}_{t-1} + (1 - \beta) \vec{g}_t^{\text{clip}}$.
 Set $\vec{w}_{t+1} = \vec{w}_t - \eta d(\vec{m}_t)$.
end for

NIGT follows has a similar structure to Algorithm 1, but with one structural change: instead of evaluating the gradient at the current point $\vec{w}_t$, it first computes

$$\vec{x}_t = \vec{w}_t + \frac{\beta(\vec{w}_t - \vec{w}_{t-1})}{1 - \beta},$$

The gradient is then evaluated at $\vec{x}_t$ rather than $\vec{w}_t$. Everything downstream, the momentum update and the final parameter step, is unchanged.

Both algorithms are clipped-momentum methods with identical update mechanics for $\vec{m}_t$ and $\vec{w}_{t+1}$. The difference is in gradients. Algorithm 1 takes a gradient at the current point, while Algorithm 2 takes it at an extrapolated point built on the momentum.

The tradeoff is one extra vector computation per step ($\vec{x}_t$) and a slightly more complex implementation, in exchange for potentially better convergence behavior.

3.2.3 Adam

Algorithm 3 The Basic Adam Optimizer

Input: Initial parameter θ_0 , step size η , batch size N_B , exponential decay rates $\beta_1, \beta_2, \varepsilon$, dataset $\{(x_i, y_i)\}_{i=1}^N$:

Set $m_0 = 0, v_0 = 0$.

for $t = 1 \dots T$ **do**

 Draw random batch $\{(x_{ik}, y_{ik})\}_{k=1}^{N_B}$ from dataset.

 Set $g_t \leftarrow \sum_{k=1}^{N_B} \nabla \ell(\{(x_{ik}, y_{ik})\}, \theta_{t-1})$.

 Set $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$.

 Set $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$.

 Set $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$.

 Set $\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$.

end for

return final parameter θ_T .

Adam is an optimizer that combines momentum with an adaptive learning rate. It adapts both the step direction (via momentum) and the step size (via second-moment normalization) automatically over the course of training.

3.3 Learning rate scheduling

An important part of the optimization process discussed in the original paper is the use of burn-in and warm-up techniques along with the decay mechanism. Theoretical analysis suggests that, while the gradient estimate may be far from the true gradient at early stages of training, it's always close to the true gradient after a number of steps. This urged authors to compare the popular *warmup* technique with a new alternative – *burn-in*.

The idea behind warmup is to increase the learning rate linearly until a chosen step T_0 , in which it attains its maximal value. After that, starting from T_1 (sometimes $T_0 = T_1$), the

learning rate is decreased in a polynomial manner (in this case, linearly until the final step T).

Burn-in follows a similar pattern with one difference – instead of a linear increase, the learning rate stays equal to 0 for T_0 steps and then increases to its maximal value until T_1 , after which decay starts.

The dynamics of learning rate in case of warmup and burn-in are presented in Figure 1.

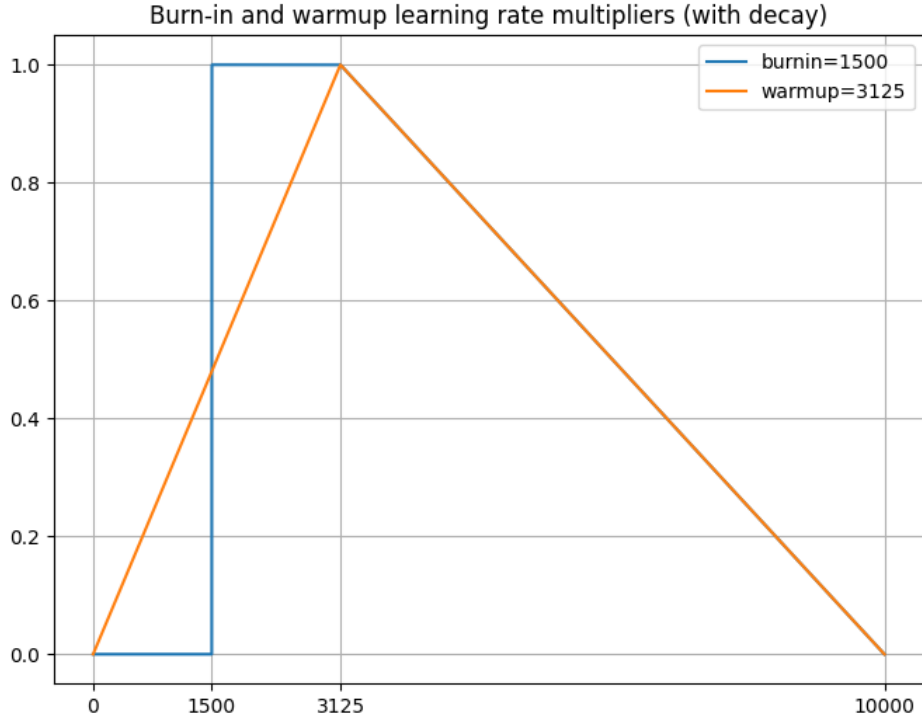


Figure 1: Example learning rate schedules. For burn-in, $T_0 = 1500$, while for warmup $T_0 = 3125$. In both cases, $T_1 = 3125$, while the amount of steps $T = 10000$.

4 Experiments

This section describes the experimental methodology and model architectures utilized to evaluate our optimization algorithms. We detail our initial attempts with bidirectional models, and the subsequent transition to custom-scaled causal language models (GPT variants).

4.1 BERT

We initially chose the BERT (Bidirectional Encoder Representations from Transformers) architecture to keep our experiments closely aligned with the approach set by [3]. In the original study, the authors tested their optimization techniques by pre-training a massive BERT-Large model [4] from scratch. This is a highly demanding task: it involves optimizing over 340 million parameters for Masked Language Modeling (MLM) across huge text datasets, which typically requires industrial computing clusters.

Given our hardware constraints, an exact replication was simply out of reach. Because of this, we decided to take a more practical and resource-friendly route right from the start. Instead of pre-training a model from scratch, we focused on fine-tuning, which is a transfer learning process where a pre-trained model is further trained on a specialized dataset to adapt its generalized language representations for a specific downstream task.

We also scaled down the architecture itself, using a smaller, BERT-Base model. While significantly lighter than its "Large" counterpart, BERT-Base remains a powerful architecture, comprising 12 transformer layers, 12 attention heads, and approximately 110 million parameters.

However, we quickly discovered that even this scaled-down approach was surprisingly heavy. Fine-tuning BERT-Base still requires a significant amount of memory and time, especially when running the multiple iterations needed to properly evaluate different optimizers.

4.2 GPT

To preserve the *pre-training from scratch* approach used by the authors, we transitioned our experimental framework to an even smaller model based on the GPT-2 architecture [1].

4.2.1 SmallGPT

Firstly, we adopted a heavily scaled-down GPT-2 configuration (a **SmallGPT**). Text processing was handled by the standard GPT-2 Byte-Pair Encoding (BPE) tokenizer (vocabulary size of 50,257), with the End-Of-Sequence (EOS) token repurposed as the padding token for efficient batching.

Architecturally, the model was constrained to a hidden embedding dimension of 256, 4 transformer layers, and 4 attention heads. The maximum sequence length was restricted to 128 tokens to mitigate the quadratic memory footprint of the self-attention mechanism. In total, this compact configuration comprises approximately 16 million trainable parameters.

The network was pre-trained using a standard Causal Language Modeling (CLM) objective. Crucially, to explicitly evaluate the optimizers under the heavy-tailed gradient conditions central to this study, artificial Cauchy noise was injected during the training process across all experiments.

4.2.2 MediumGPT

After a few runs on the **SmallGPT** model, we tried transitioning to a slightly larger **MediumGPT** architecture. While the context window remained at 128 tokens, the model's capacity was expanded to an embedding dimension of 384, 8 transformer layers, and 6 attention heads. This upgrade increased the total parameter count to approximately 33.5 million. The primary goal of this change was to provide a richer, more expressive parameter space for the optimizers to navigate and potentially introduce heavier tails, while remaining computationally feasible for our hardware.

4.2.3 WikiText Dataset

To train and fine-tune our models, we utilized the WikiText dataset [5], which is a high-quality corpus extracted exclusively from verified articles on Wikipedia. It provides over 100 million tokens of complex vocabulary, proper grammar, and long-term contextual dependencies. This made it an excellent benchmark for testing how well our optimizers could handle real-world language modeling without the unpredictability of low-quality internet text.

4.2.4 Grid search

In the original paper authors performed a learning rate grid search in order to compare optimizers fairly. In accord with this, we also test various learning rates for all three algorithms separately, although our grid is much less dense than the authors'. For each learning rate value, we trained a model with 3125 warmup steps in order to determine which one is the best. For chosen learning rates, the next experiment is conducted.

4.2.5 Burn-in and warmup comparison

After choosing optimal learning rates, we conducted the key experiment – comparison of various burn-in configurations with warmup. Once again, we performed it for each algorithm separately in order to obtain meaningful results.

4.2.6 Artificial heavy-tail data

Another thing we decided to investigate was how artificial heavy-tailed data might change the results of these algorithms. Our idea was to create a function that would sample data based on the t-student distribution,

$$X \sim t_{[2]},$$

and make only a certain subset of samples relevant to the label calculated based upon it. We sample

$$w \sim U_{[0.5, 2]},$$

where w is a vector of weights of length equal to half of the number of variables. We then fill it with zeros, making w_0 , which is the same length as the number of variables. Finally we calculate

$$X \cdot w_0 + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, 0.5)$, and then we fed it through the sigmoid function, creating the label vector Y .

That change required us to change the shape of the model. We had to adjust both the way the data was injected, since it was not longer in the form of text undergoing tokenization and prediction, as well as the prediction returned, which we changed to be a binary label. Our justification was that maybe, since the model is smaller than the one used in the main article [3], we can recreate the results in a ‘roundabout’ way.

5 Results

5.1 BERT

Attempting to directly apply the burn-in and warm-up hyperparameters from the original paper to a fine-tuning (rather than pre-training) scenario with Alg1 optimizer failed to reproduce the authors’ results. Figure 2a demonstrates that the training loss decreased steadily only during the initial 10,000 steps, without ever rising. Figure 2b show only a marginal accuracy improvement. Moreover, the results remained remarkably consistent across all evaluated burn-in and warm-up scheduling variants indicating no meaningful difference across configurations.

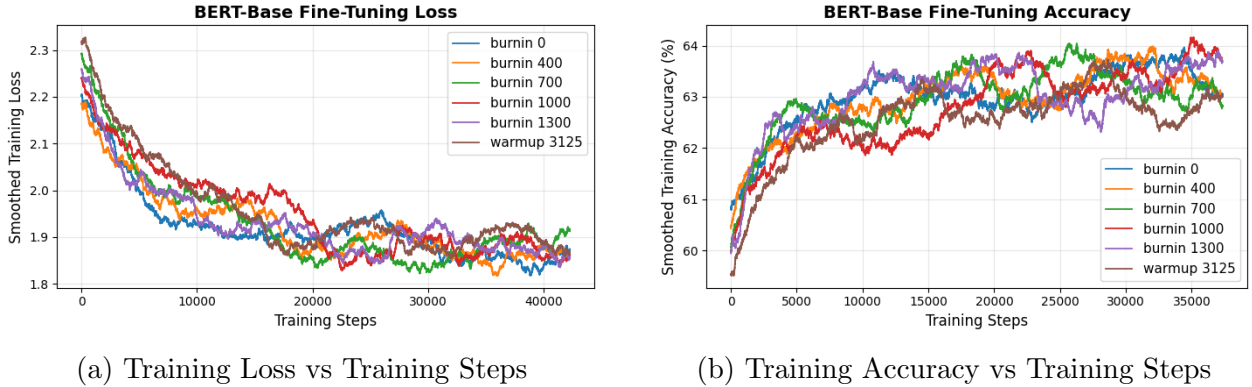


Figure 2: BERT-Base fine-tuning, scores smoothed across 5000 steps.

	Alg1 B-0	Alg1 B-400	Alg1 B-700	Alg1 B-1000	Alg1 B-1300	Alg1 W-3125
mean	1.937	1.941	1.954	1.967	1.951	1.975
std	0.077	0.076	0.100	0.094	0.081	0.104
min	1.818	1.817	1.821	1.828	1.835	1.853
max	2.206	2.189	2.293	2.241	2.260	2.328

Table 1: Summary statistics for training loss on fine-tuning BERT-Base.

	Alg1 B-0	Alg1 B-400	Alg1 B-700	Alg1 B-1000	Alg1 B-1300	Alg1 W-3125
mean	62.856	62.843	62.770	62.668	62.856	62.382
std	0.979	1.067	1.235	1.323	1.440	1.324
min	59.668	57.259	57.329	56.053	55.842	56.977
max	64.846	65.792	65.328	66.346	65.410	65.675

Table 2: Summary statistics for training accuracy on fine-tuning BERT-Base.

5.2 GPT

5.2.1 SmallGPT baseline

Moving to **SmallGPT** architecture, we first compared different optimizers including the original SGD and Adam as well as Alg1 and Alg2 proposed in the original paper. We tested the original ones without any burn-in or warmup and with 400 steps warmup. The Alg1 and Alg2 were tested with 400 steps burn-in and warmup. Note that we used $lr = 3e - 4$ across all these experiments.

The loss curves (Figure 3a) show superiority of the Adam optimizer, and similar performance of Alg1, Alg2 and SGD optimizers. These finding continue in the accuracy plot (Figure 3b) where the Adam optimizer achieved the highest score, with Alg1 and Alg2 having lower scores but still outperforming SGD.

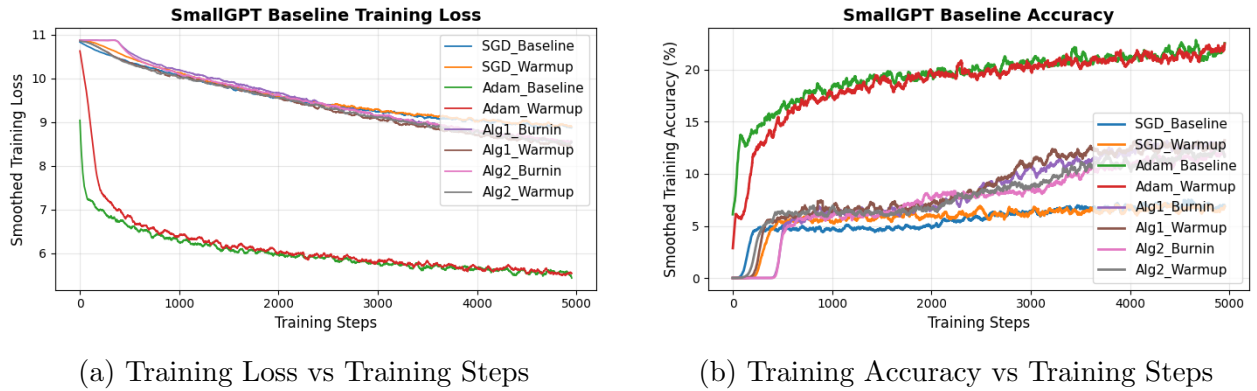


Figure 3: SmallGPT pre-training, scores smoothed across 50 steps.

	SGD	SGD W-400	Adam	Adam W-400	Alg1 B-400	Alg1 W-400	Alg2 B-400	Alg2 W-400
count	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000
mean	9.542	9.588	6.021	6.161	9.516	9.407	9.499	9.427
std	0.550	0.575	0.572	0.844	0.727	0.686	0.703	0.662
min	7.119	7.687	4.034	3.037	7.598	7.683	7.927	7.801
max	10.877	10.907	10.862	10.908	10.919	10.902	10.905	10.892

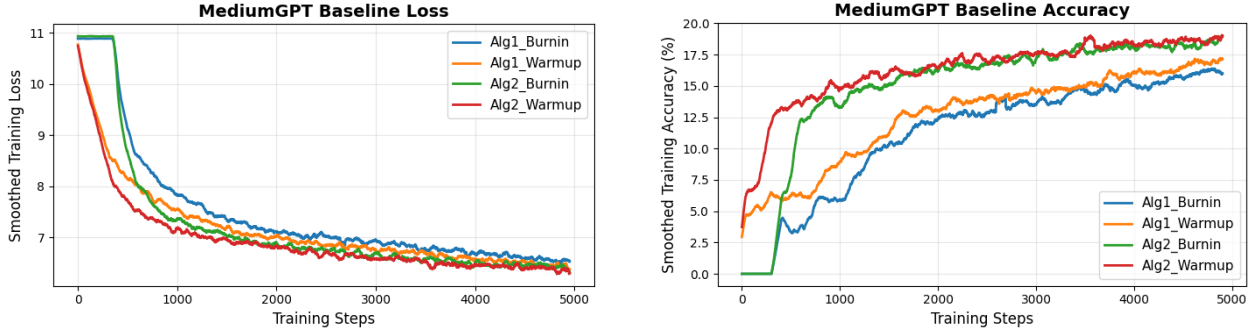
Table 3: Summary statistics for training loss on fine-tuning SmallGPT.

	SGD	SGD W-400	Adam	Adam W-400	Alg1 B-400	Alg1 W-400	Alg2 B-400	Alg2 W-400
count	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000	5001.000
mean	5.590	5.738	19.416	18.769	8.296	8.830	7.690	8.145
std	1.870	2.066	3.474	4.306	4.061	3.790	3.588	3.295
min	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
max	33.750	29.412	42.969	65.385	32.558	45.455	23.737	25.980

Table 4: Summary statistics for training accuracy on fine-tuning SmallGPT.

5.2.2 MediumGPT baseline

Increasing size of the GPT architecture did not show any meaningful difference in the training process. The loss curves shown in Figure 4a dropped significantly only in the first 1000 steps and plateaued. There is no significant difference between both optimizing algorithms and burnin/wamup setups. The accuracy plot (Figure 4b) show that the Alg2 performed a bit better than Alg1, with no difference between burn-in/warmup strategies. All experiments used $lr = 3e - 4$.



(a) Training Loss vs Training Steps

(b) Training Accuracy vs Training Steps

Figure 4: MediumGPT pre-training, scores smoothed across 50 steps.

	Alg1 B-400	Alg1 W-400	Alg2 B-400	Alg2 W-400
mean	7.472	7.165	7.213	6.957
std	1.216	0.876	1.251	0.850
min	2.725	3.918	5.459	2.743
max	11.012	10.958	10.981	10.921

Table 5: Summary statistics for training loss on fine-tuning MediumGPT.

	Alg1 B-400	Alg1 W-400	Alg2 B-400	Alg2 W-400
mean	11.083	12.520	14.910	16.186
std	5.281	4.375	5.503	3.893
min	0.000	0.000	0.000	0.000
max	74.194	55.932	36.111	66.667

Table 6: Summary statistics for training accuracy on fine-tuning MediumGPT.

5.2.3 Grid search

The grid search results show that the optimal learning rate for each algorithm was different than we assumed at the beginning. The detailed plots are available in Appendix A.

It turned out that for both Alg1 and Alg2 the learning rate that led to the best validation accuracy was $1e-2$, meanwhile Adam was performing the best in case of learning rate equal to $1e-3$.

We can see that high learning rate leads to gradient explode and total training destabilization. Interestingly, among remaining configurations, the average gradient norm doesn't differ much. It also seems not to depend on the training step. Lastly, accuracy on the validation set seems to reflect loss, so there is no overfitting present.

Results for Alg2 yield the same optimal learning rate as for Alg1. One key difference is the gradient norm, which is two orders of magnitude higher than in case of Alg1. Other than that, it seems that accuracy doesn't grow as steadily – it stays on a plateau for a long time (and, in some cases, doesn't improve).

Adam consistently achieves higher accuracy than the remaining two optimizers. For higher learning rates, we can see the same results as in case of Alg1 – sharp loss decrease followed by a slow increase. Moreover, gradient norms are similar to those achieved by Alg2.

5.2.4 Burn-in and warmup comparison

After finding optimal learning rates for each algorithm, we checked how different burn-in step amounts influence the training process. We tested the following values, taken from the original paper:

- Burn-in value of 0 (instant increase to maximum)
- Burn-in value of 250
- Burn-in value of 500
- Burn-in value of 1000
- Burn-in value of 2000
- Burn-in value of 3125
- Warmup value of 3125 (for comparison)

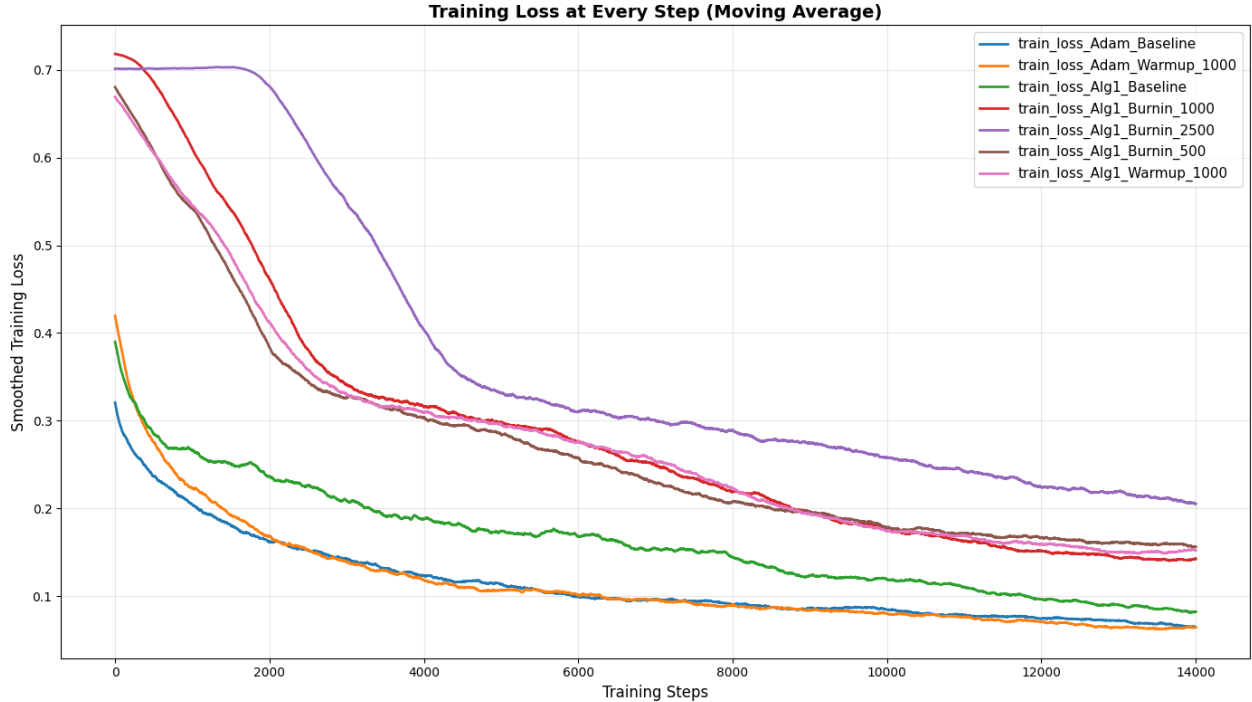
We tested these values (with the learning rate decay starting at 3125 steps) for each algorithm and included results in Appendix B.

The main phenomenon we can observe is how both burn-in and warmup seem to delay the process of training. Regardless of how many steps are used for burn-in, the loss function plunges down as soon as it finishes the burn-in process. On the other hand, in case of warmup, the loss function is decreased immediately, although to a lesser extent than in case of no technique applied (burnin=0). This leads to a conclusion that in this case these techniques do not result in stable and effective training, contrary to what authors achieved in the original paper using BERT-Large.

5.2.5 Artificial heavy-tailed data

Artificial heavy-tailed data did not provide the expected results. Just as in the case of other configurations in our project, Alg1 did not outperform the Adam optimizer, just as the original paper [3] would suggest.

Additionally, after noticing that the Alg1 performed so poorly, we did not repeat the experiment for Alg2 due to time constraints.



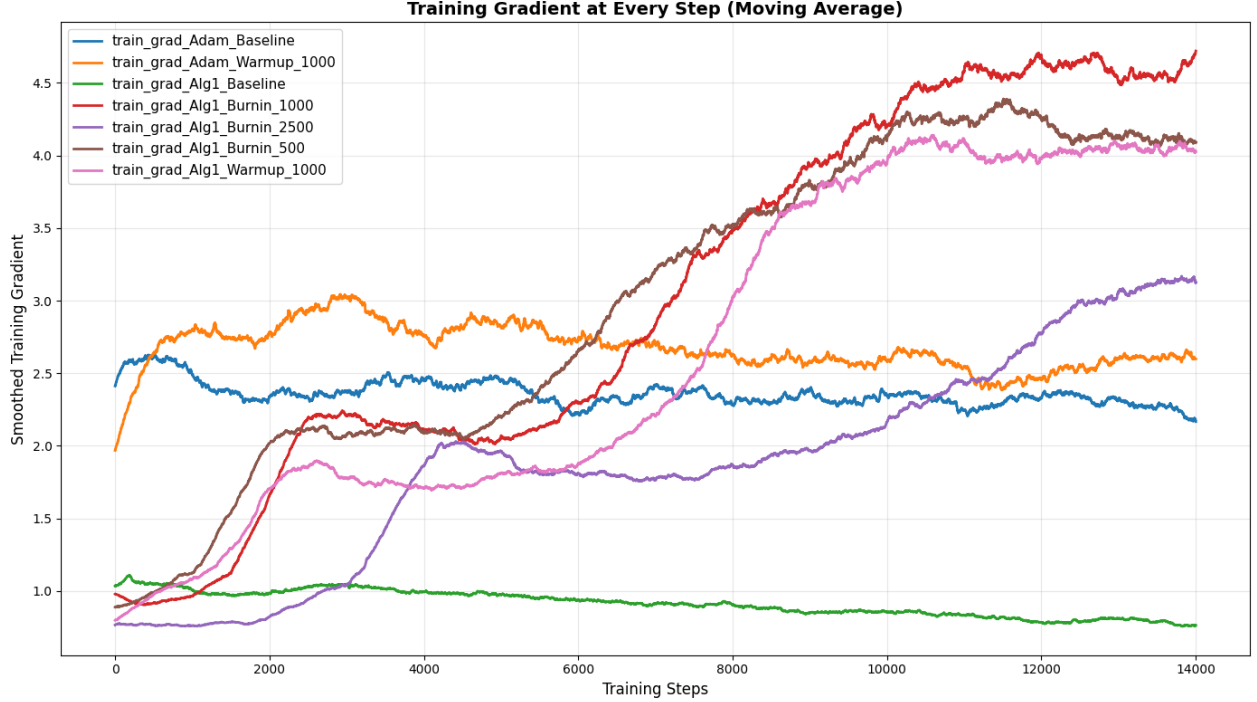
	Adam	Adam W-1000	Alg1	Alg1 B-1000	Alg1 B-2500	Alg1 B-500	Alg1 W-1000
mean	0.121	0.127	0.170	0.297	0.379	0.283	0.287
std	0.098	0.119	0.124	0.190	0.197	0.171	0.171
min	0.000	0.000	0.000	0.009	0.030	0.011	0.009
25%	0.053	0.049	0.083	0.155	0.224	0.157	0.158
50%	0.095	0.093	0.140	0.247	0.320	0.240	0.248
75%	0.160	0.162	0.223	0.388	0.537	0.363	0.376
max	0.839	0.842	0.909	0.832	0.856	0.902	0.824

Table 7: Summary statistics for training loss on synthetic data.

As we can see, when it came to loss during training, Adam managed to score the lowest result, eventually fighting for first place with its own warmup version. Third place was taken by the Algorithm 1 baseline, with a mean difference of 0.05 between the previous place.

Algorithm 1 with various burn-in implementations scored awfully, with the next best place achieving on average a 1.1 higher loss.

In case of training gradient, the situation is a bit more complicated. As we can see above, the outcome varies greatly between the algorithms and their various implementations.



	Adam	Adam W-1000	Alg1	Alg1 B-1000	Alg1 B-2500	Alg1 B-500	Alg1 W-1000
mean	2.349	2.639	0.912	3.015	1.879	2.963	2.652
std	1.802	2.114	0.640	2.461	1.436	2.427	2.138
min	0.002	0.003	0.001	0.196	0.227	0.236	0.222
25%	1.020	1.083	0.445	1.235	0.876	1.225	1.141
50%	1.862	2.048	0.744	2.269	1.417	2.210	2.014
75%	3.157	3.589	1.212	4.005	2.457	3.940	3.441
max	14.389	20.578	8.185	22.234	12.817	20.418	17.382

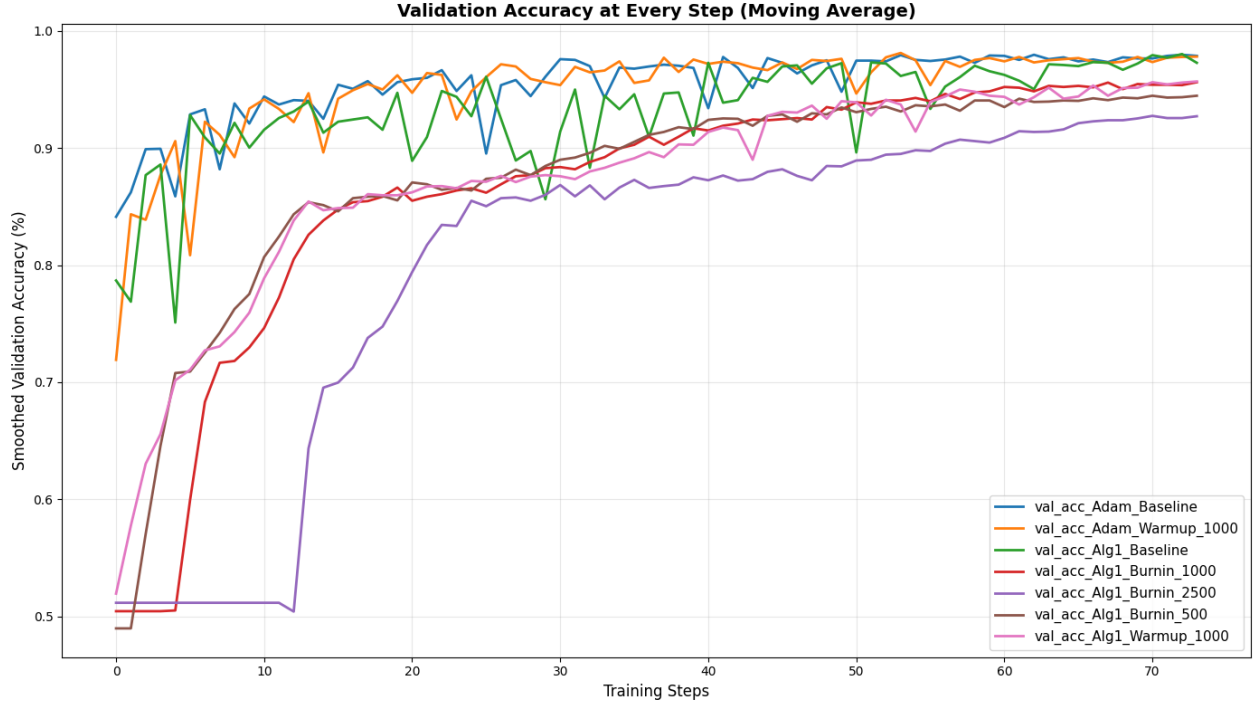
Table 8: Summary statistics for training gradient on synthetic data.

When it came to Adam and Adam with warmup, the gradients are more or less stable. For Algorithm 1, the gradients are stable as well, but lower. This is to be expected, as Alg1 uses clipping.

Things complicate when we investigate Alg1 with burn-in and warmup. In all cases, the gradient grows steadily, achieving the best gains for the burn-in 1500.

When it comes to the accuracy on the synthetic heavy-tailed model, all of the various algorithm implementations scored quite well. As we can see, Both Adam implementations and the baseline Alg1 implementation were head in head in their respective accuracies. The burn-in versions and the warmup version of Algorithm 1, however, scored a little bit below the three. Our theory for why that is boils down to the fact, that the burn-in implementations for Algorithm 1 simply did not have enough time to "catch up" to both Adams and the baseline Alg1. Alternatively, the case could be that it would never truly "catch up", as it seems that the burn-in implementation fails for synthetic data.

All of this leads us to conclude, that just as was the case with previous experiments, the



	Adam	Adam W-1000	Alg1	Alg1 B-1000	Alg1 B-2500	Alg1 B-500	Alg1 W-1000
mean	0.955	0.951	0.933	0.864	0.802	0.874	0.874
std	0.030	0.044	0.045	0.121	0.147	0.098	0.092
min	0.841	0.719	0.751	0.504	0.504	0.490	0.519
25%	0.945	0.948	0.914	0.856	0.753	0.859	0.860
50%	0.968	0.966	0.945	0.906	0.869	0.912	0.892
75%	0.975	0.974	0.967	0.942	0.898	0.935	0.940
max	0.980	0.981	0.980	0.956	0.927	0.945	0.957

Table 9: Summary statistics for validation accuracy on synthetic data

original paper [3] does not hold for synthetic data on the smallGPT architecture. This, again, shows how strict the conditions are for these mathematical principles to actually, truly work.

As mentioned before, after seeing these results and the mediocre-to-poor performance of all the burn-in implementations for Algorithm 1, we’ve decided not to go forward with repeating the synthetic heavy-tailed data experiment for the second algorithm introduced in [3], due to time limitations.

6 Conclusions

In this project, we set out to evaluate Algorithm 1 (Normalized SGD with Clipping and Momentum) and Algorithm 2 (NIGT with Clipping) from [3] against the Adam optimizer, with a particular focus on the role of burn-in and warmup strategies for gradient stabilization. We specifically wanted to check the results obtained in the original paper [3] against smaller, simpler models. This was, predictably, dictated by our limitations in hardware and time.

As computational constraints reshaped our experimental design, our initial attempt to closely replicate the original paper’s setup using BERT-Large proved infeasible given our hardware. Even the scaled-down BERT-Base model required substantial time and memory to fine-tune. This led us towards fully training smaller architectures, like SmallGPT and MediumGPT, trained from scratch on WikiText. This better preserved the *pre-training* setting of the original study while being achievable on the hardware we’ve had access to.

As it turns out, when the strict mathematical constraints aren’t met, like in the original paper [3], **Adam consistently outperformed Alg1 and Alg2**. Contrary to the convergence advantages claimed in [3], Adam (with and without warmup) achieved the lowest training loss and most stable gradient behavior across nearly all of our experimental settings, including on the synthetic heavy-tailed dataset. Alg1 and Alg2 were broadly comparable to each other in performance, but both consistently trailed behind Adam, often by a substantial margin, especially once burn-in was introduced. In general, burn-in and warmup did not help in our experiments. Rather than improving convergence, applying burn-in to Alg1 noticeably hurt performance relative to the no-burn-in baseline, an effect that grew more pronounced with longer burn-in windows (e.g. burn-in 2500 performed worse than burn-in 500 or 1000). We hypothesize that for a model as small as SmallGPT, skipping initial training steps before gradient accumulation mainly costs the optimizer useful early learning signal without providing a compensating benefit. This suggests that the burn-in/warmup techniques described in [3] may be only beneficial at the scale of much larger transformer models such as BERT-Large, and do not transfer to smaller architectures.

Synthetic heavy-tailed data did not reproduce the paper’s claims either. Our artificial heavy-tailed data experiments, designed to isolate the effect of heavy-tailed gradient noise outside the language modeling setting, repeated the previous results. Alg1 did not outperform Adam, and the gap was, if anything, more pronounced than in our language modeling experiments.

Overall, our results suggest that the theoretical advantages of clipped, momentum-based, normalized methods used on heavy-tailed gradient noise, while rigorously argued in the original paper [3], do not translate to smaller model scales. Whether this is a fundamental limitation, an artifact of insufficient training, or a hyperparameter sensitivity issue remains an open question that we believe warrants further investigation, especially at intermediate model scales between our SmallGPT/MediumGPT setup and the original BERT-Large benchmark.

References

- [1] Daixuan Cheng, Yuxian Gu, Shaohan Huang, Junyu Bi, Minlie Huang, and Furu Wei. Instruction pre-training: Language models are supervised multitask learners, 2024.
- [2] Ashok Cutkosky and Harsh Mehta. Momentum improves normalized sgd, 2020.
- [3] Ashok Cutkosky and Harsh Mehta. High-probability bounds for non-convex stochastic optimization with heavy tails, 2021.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [5] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.
- [6] Yurii Nesterov. A method for solving the convex programming problem with convergence rate $o(1/k^2)$. *Proceedings of the USSR Academy of Sciences*, 269:543–547, 1983.
- [7] B.T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- [8] Jingzhao Zhang, Sai Praneeth Karimireddy, Andreas Veit, Seungyeon Kim, Sashank J Reddi, Sanjiv Kumar, and Suvrit Sra. Why are adaptive methods good for attention models?, 2020.

Appendices

A Grid search plots

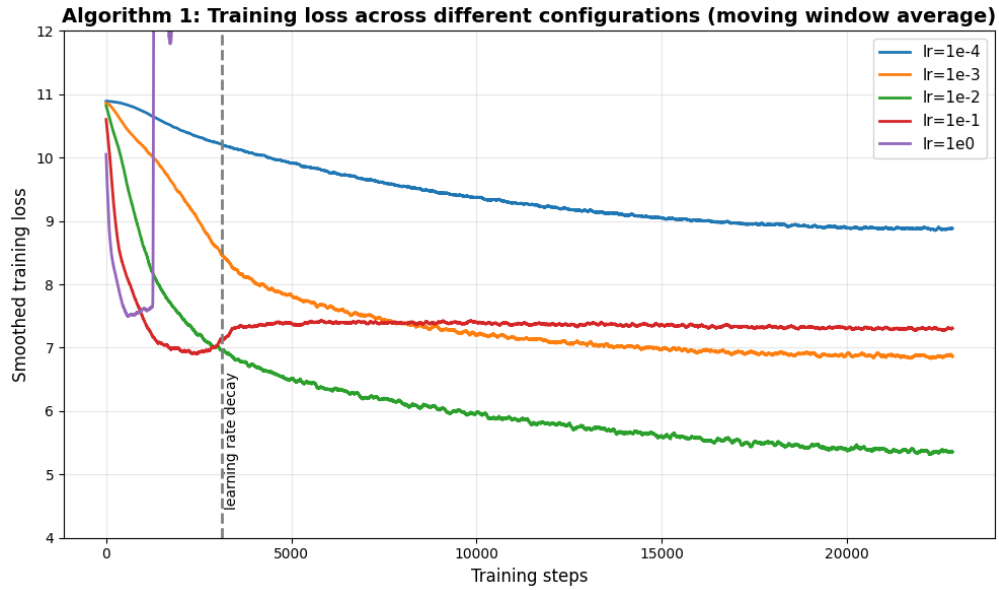


Figure 5: Training loss for different settings of Alg1.

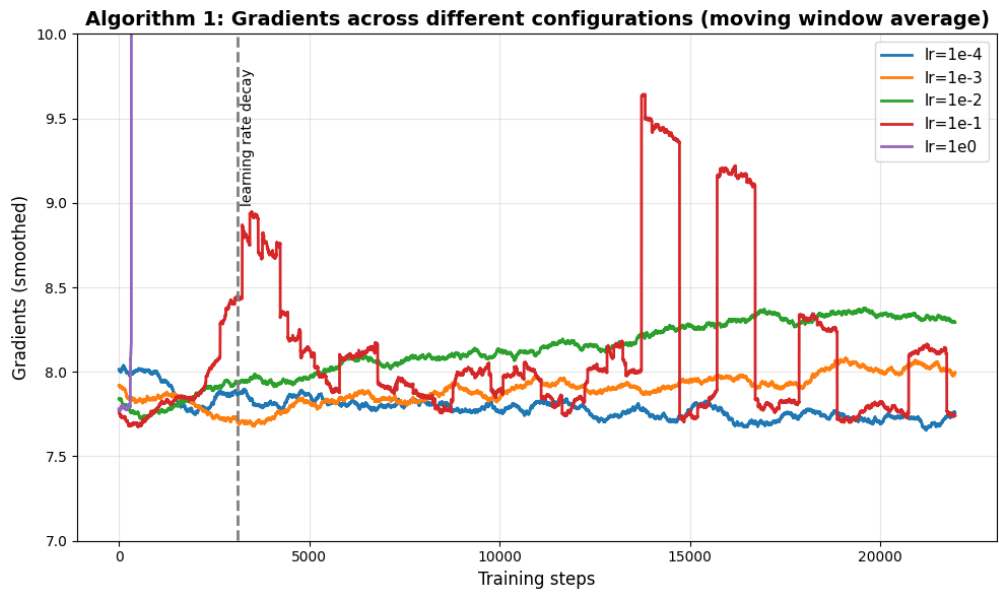


Figure 6: Gradients across training for different settings of Alg1.

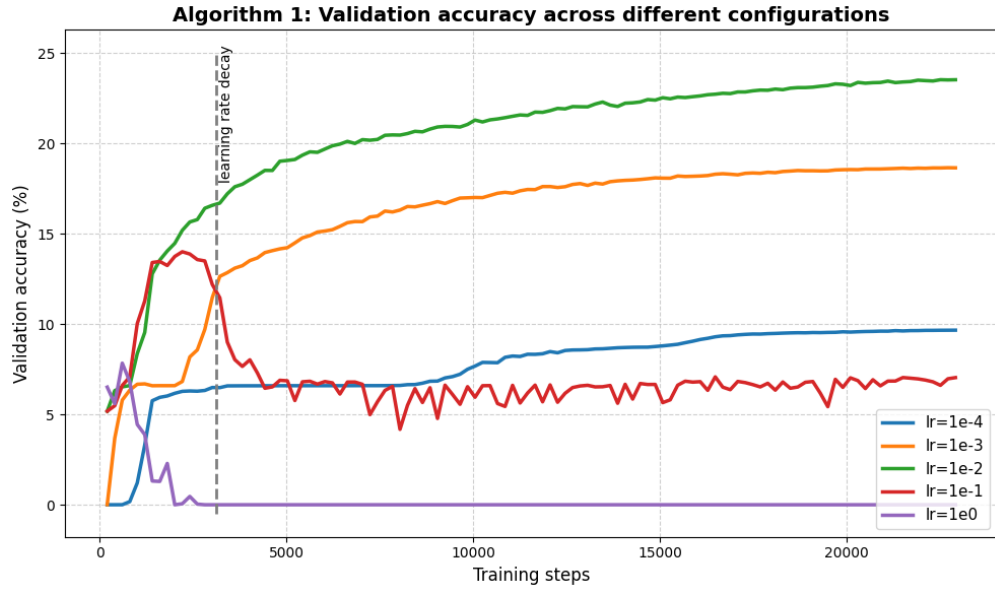


Figure 7: Validation accuracy for different settings of Alg1.

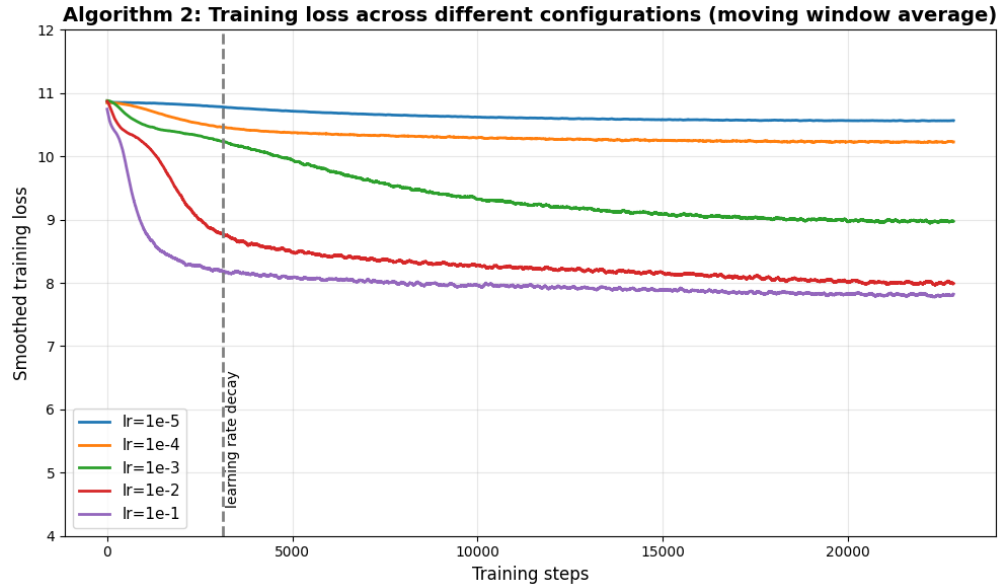


Figure 8: Training loss for different settings of Alg2.

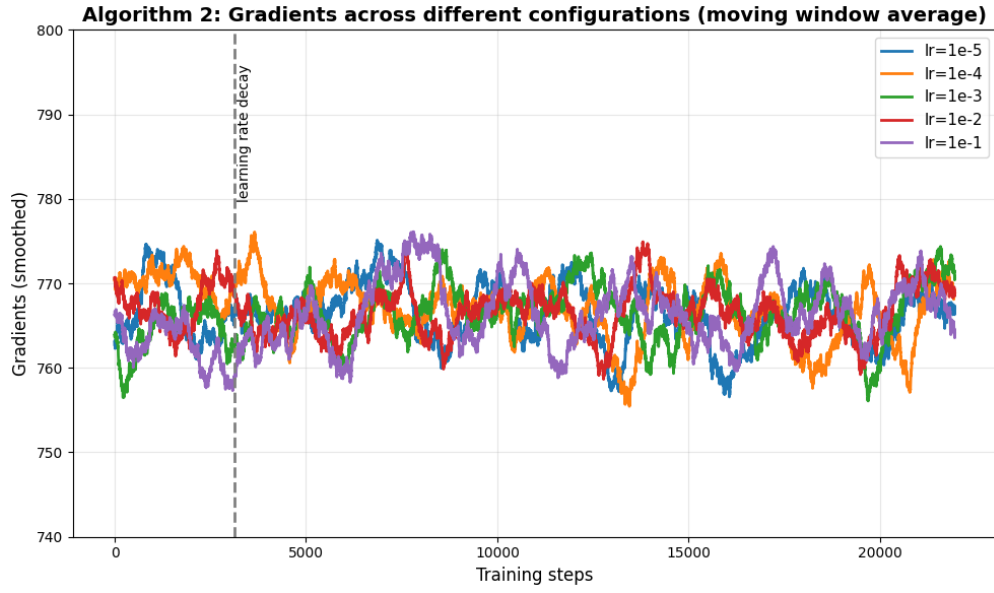


Figure 9: Gradients across training for different settings of Alg2.

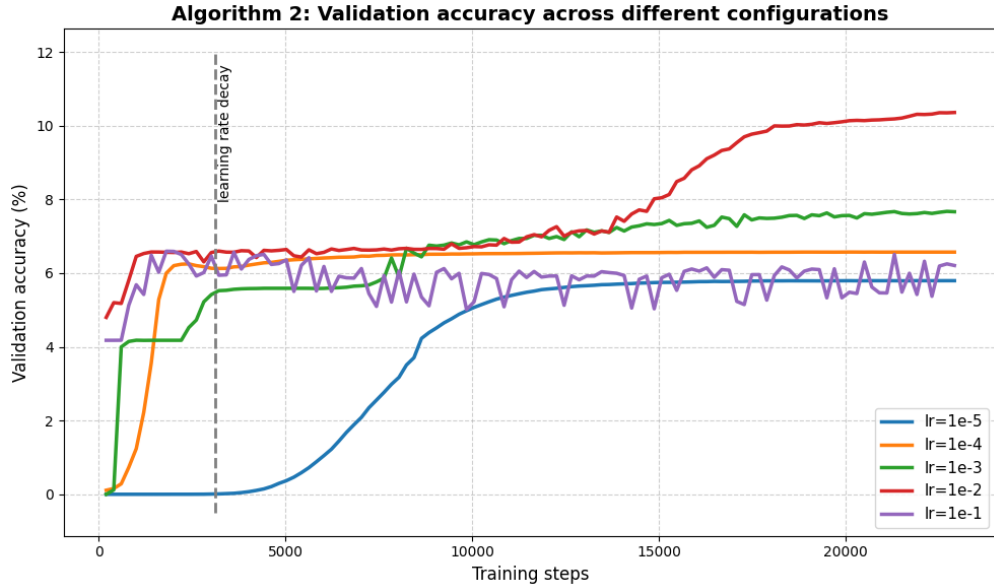


Figure 10: Validation accuracy for different settings of Alg2.

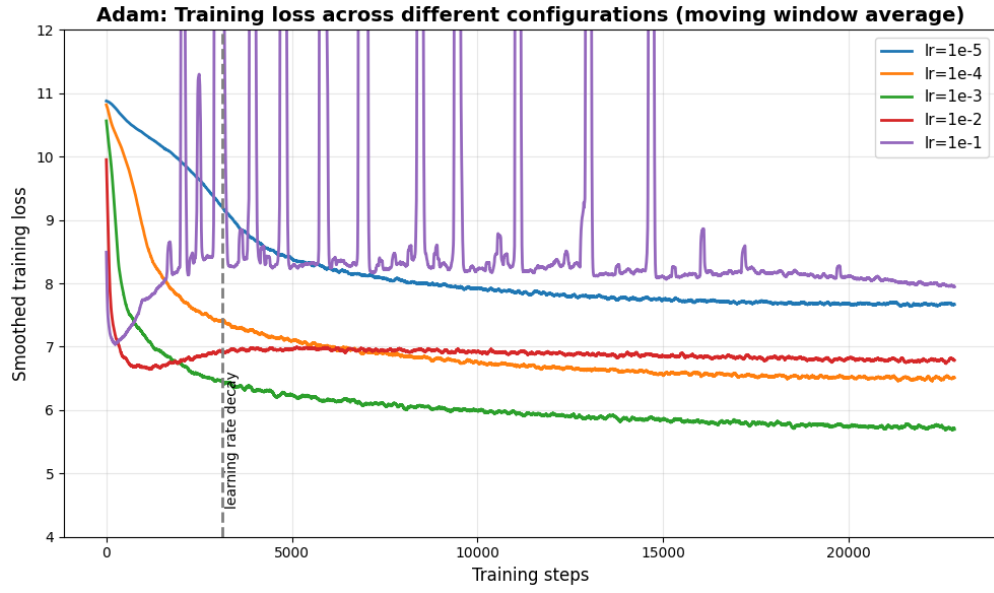


Figure 11: Training loss for different settings of Adam.

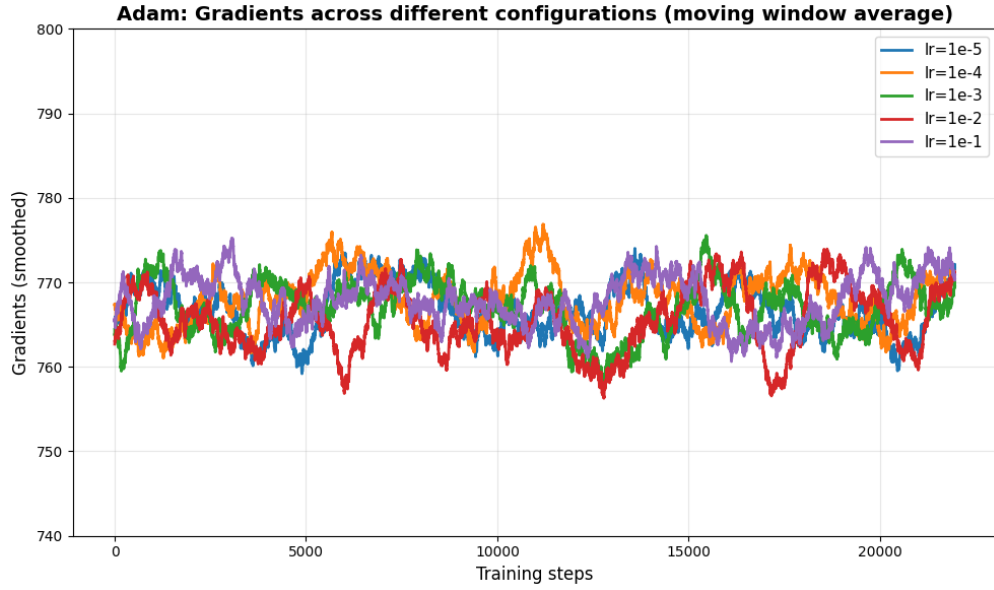


Figure 12: Gradients across training for different settings of Adam.

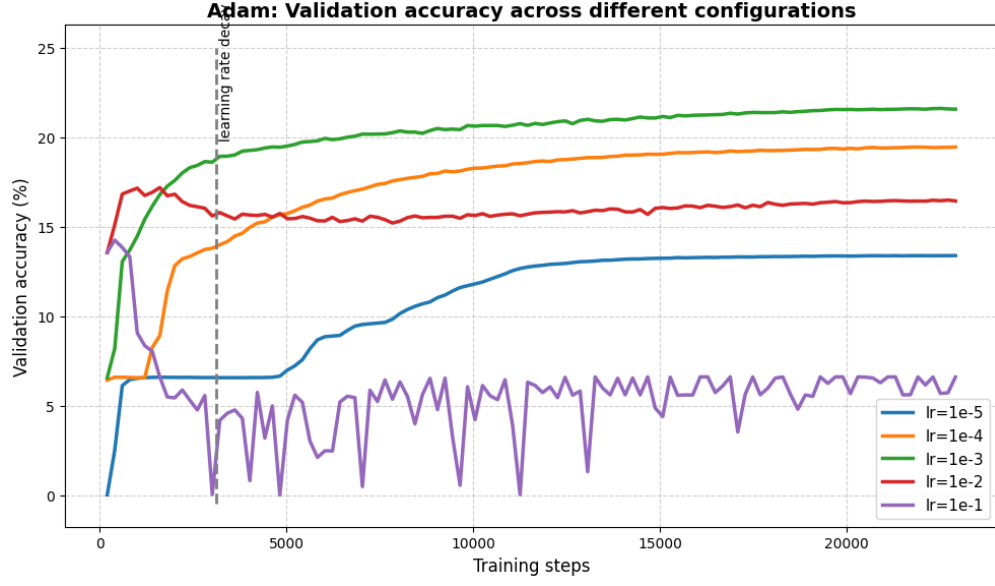


Figure 13: Validation accuracy for different settings of Adam.

B Burn-in and warmup plots



Figure 14: Training loss for different settings of Alg1.

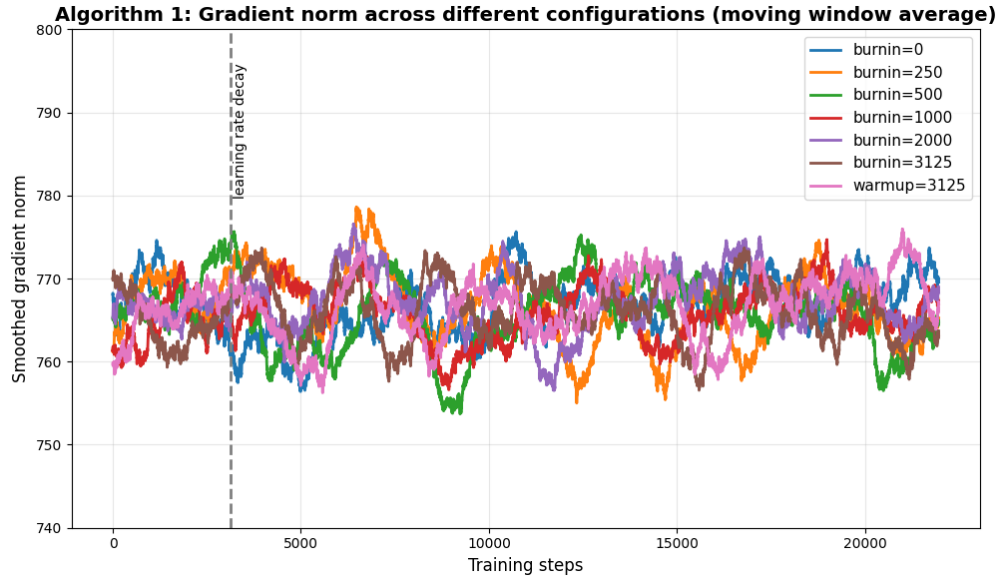


Figure 15: Gradients across training for different settings of Alg1.

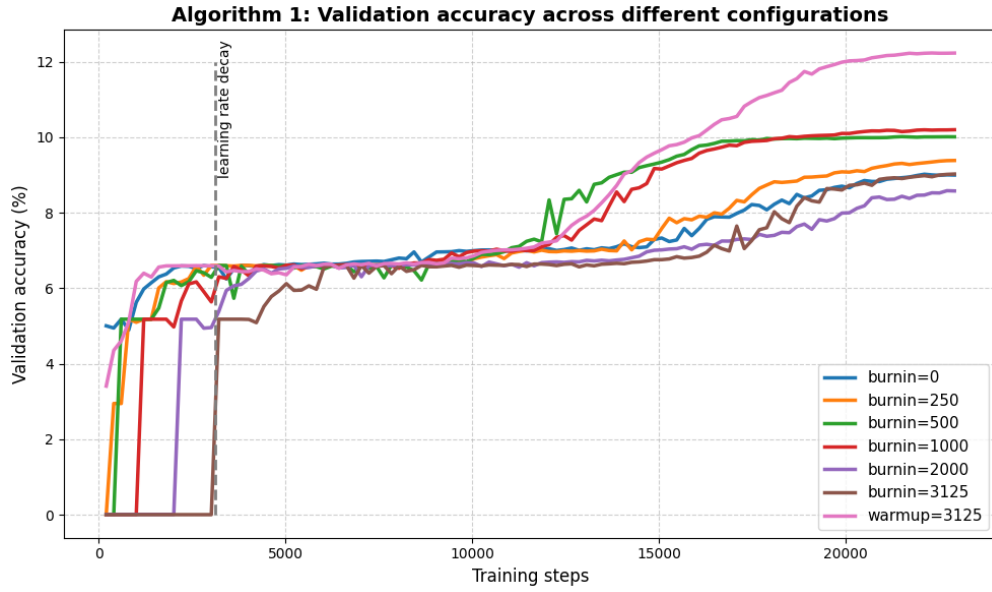


Figure 16: Validation accuracy for different settings of Alg1.



Figure 17: Training loss for different settings of Alg2.

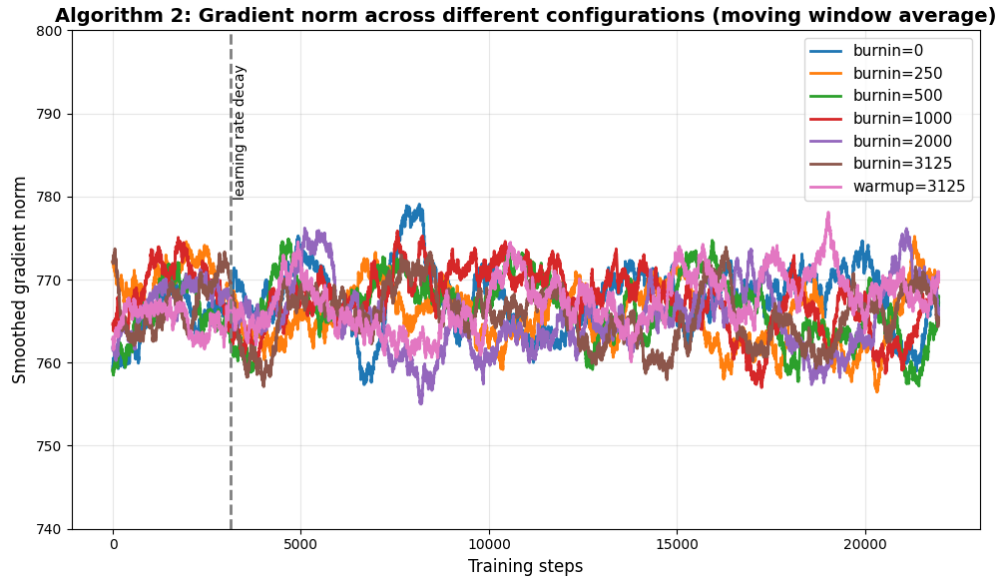


Figure 18: Gradients across training for different settings of Alg2.

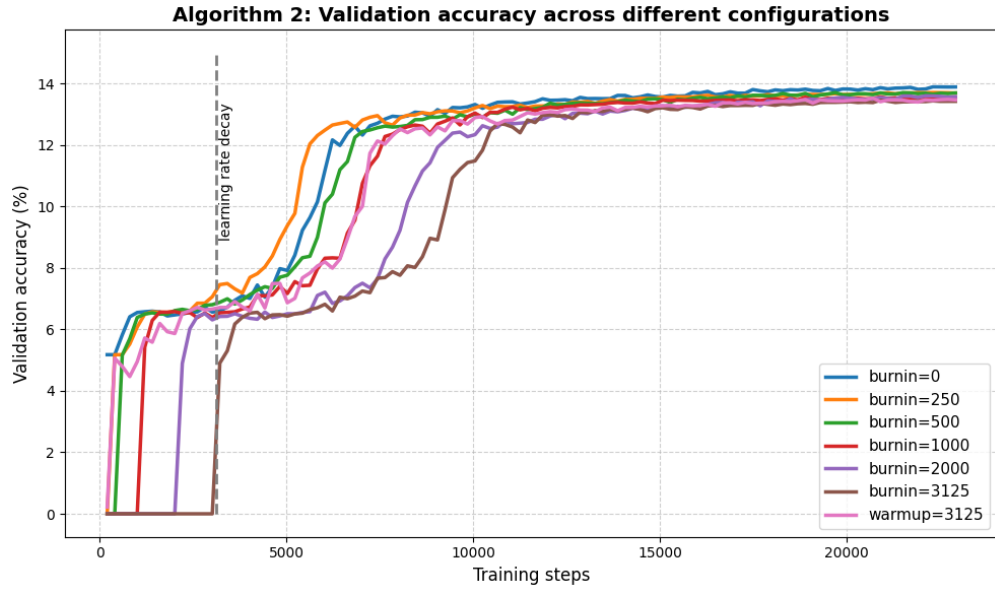


Figure 19: Validation accuracy for different settings of Alg2.

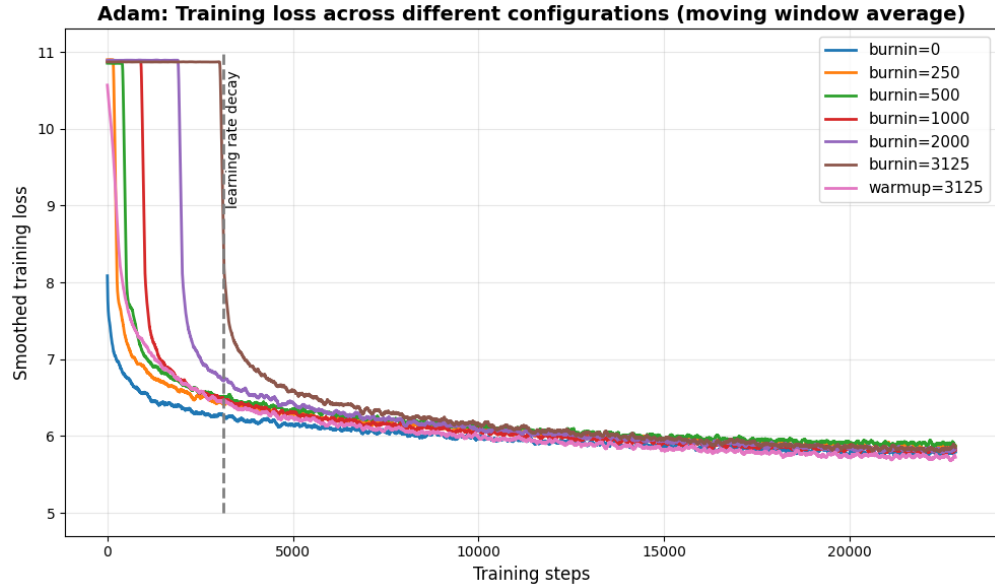


Figure 20: Training loss for different settings of Adam.

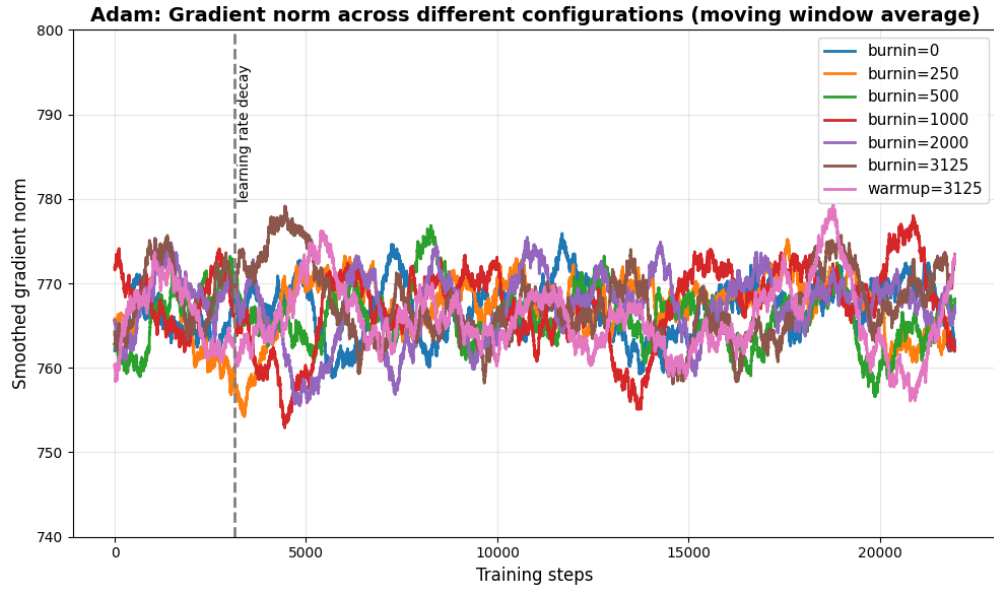


Figure 21: Gradients across training for different settings of Adam.

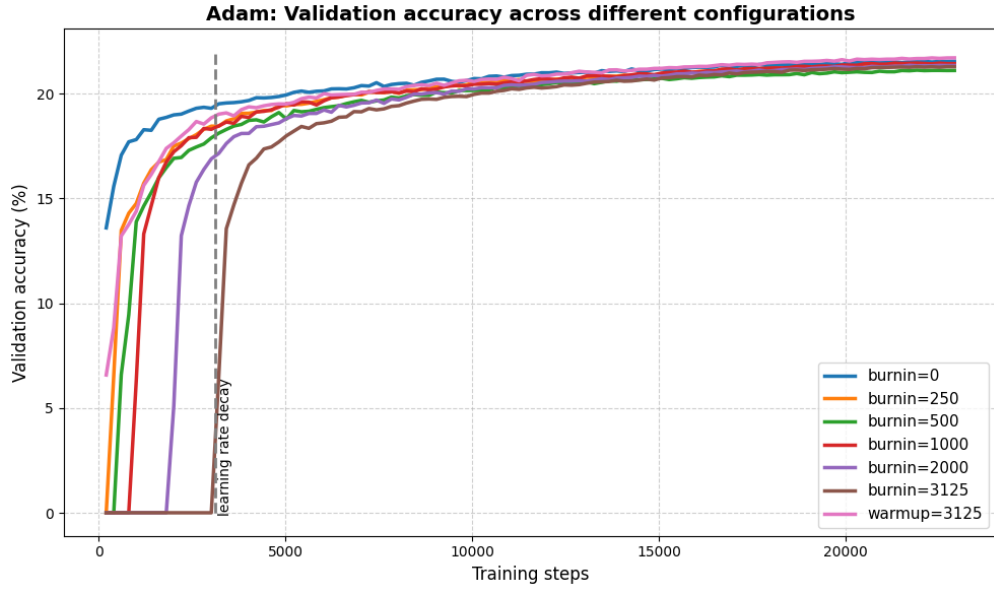


Figure 22: Validation accuracy for different settings of Adam.